

PROGRAMMAZIONE II

INTERFACCE FUNZIONALI E STREAM

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

INTERFACCIA FUNZIONALE

Un'interfaccia funzionale contiene un solo metodo

// alias **Single Abstract Method**

- ▶ Come `java.lang.Comparator`

La semantica è puramente funzionale

- ▶ L'esito del metodo si basa solo sui suoi argomenti

Le interfacce funzionali predefinite sono nel package `java.util.function`

- ▶ Versioni per tipi primitivi e generici
- ▶ È buona norma annotarle con `@FunctionalInterface`

PREDICATE

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

Una funzione che accetta un argomento di tipo `T` e restituisce un `boolean`

- Spesso usata per filtrare elementi in una collezione

```
Predicate<Integer> isEven = new Predicate<>() { // con una           1
    @Override                                // classe anonima       2
    public boolean test(Integer n) { return n \% 2 == 0; }           3
};                                           4
```

PREDICATE

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

Una funzione che accetta un argomento di tipo `T` e restituisce un `boolean`

- Spesso usata per filtrare elementi in una collezione

```
Predicate<Integer> isEven = n -> n \% 2 == 0; // con una lambda      1
                                                                    2
System.out.println(isEven.test(2)); // stampa 'true'                3
System.out.println(isEven.test(3)); // stampa 'false'                4
```

PREDICATI SPECIALIZZATI

```
@FunctionalInterface
public interface IntPredicate {
    boolean test(int v);
}
```

Sono fornite anche versioni specializzate per tipi primitivi

- Stesso utilizzo della versione con generici

```
IntPredicate isEven = n -> n \% 2 == 0;

System.out.println(isEven.test(2)); // stampa 'true'
System.out.println(isEven.test(3)); // stampa 'false'
```

1
2
3
4

FUNCTION

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

Una funzione che accetta un argomento di tipo **T** e restituisce un risultato di tipo **R**

- Spesso usata per mappare un valore in un altro

```
Function<String, Integer> strLen = str -> str.length();
System.out.println(strLen.apply("hello")); // stampa '5'
```

1
2
3

BiPREDICATE

```
@FunctionalInterface
public interface BiPredicate<T, U> {
    boolean test(T t, U u);
}
```

Una funzione che accetta due argomenti di tipo **T** e **U** e restituisce un **boolean**

- Spesso usata per verificare condizioni su due valori

```
BiPredicate<Integer, Integer> isGreaterThan = (a, b) -> a > b;           1
                                                                           2
System.out.println(isGreaterThan.test(2, 5)); // stampa 'false'          3
System.out.println(isGreaterThan.test(3, 1)); // stampa 'true'           4
```


BiFunction

```
@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
}
```

Una funzione che accetta due argomenti di tipo **T** e **U** e restituisce un risultato di tipo **R**

- Spesso usata per implementare operazioni su due valori

```
Function<Integer, Integer, Integer> sumInt = (a, b) -> a + b;
System.out.println(sumInt.apply(3, 9)); // stampa '12'
```

1
2
3

CONSUMER

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

Una funzione che accetta un argomento di tipo `T` e non restituisce alcun risultato

- Spesso usata per modellare effetti collaterali (side-effect)

```
Consumer<String> printUC = str -> System.out.println(str.toUpperCase()); 1
                                                                    2
printUC.accept("string"); // stampa 'STRING'                             3
```

SUPPLIER

```
@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

Una funzione che non accetta argomenti e restituisce un valore di tipo **T**

- Spesso usata per la generazione di valori

```
Supplier<Double> randomNum = () -> Math.random();           1
                                                                    2
System.out.println(randomNum.get()); // stampa un numero decimale casuale 3
```

ALTRE INTERFACCE FUNZIONALI

```
public interface UnaryOperator<T> extends Function<T, T>
```

- Una funzione che accetta un argomento di tipo **T** e restituisce un risultato dello stesso tipo **T**

```
public interface BinaryOperator<T> extends BiFunction<T, T, T>
```

- Una funzione che accetta due argomenti di tipo **T** e restituisce un risultato di tipo **T**

```

@FunctionalInterface
public interface Validator<T> { boolean validate(T input); }

Validator<String> noHtmlValidator = input -> {
    if (input.contains('<') || input.contains('>')) return false;
    return true;
};

```

METHOD REFERENCE

Rappresentazione compatta dell'invocazione di un'interfaccia funzionale

`System.out::println` equivalente a `str -> System.out.println(str)`

metodo statico	<code>Class::staticMethod</code>
metodo d'istanza (parametri)	<code>object::instanceMethod</code>
metodo non statico (senza parametri)	<code>Class::nonStaticMethod</code>
costruttore	<code>Class::new</code>

METHOD REFERENCE (CONT.)

Metodo d'istanza di un oggetto specifico

```
String hexDigits = "0123456789ABCDEF";           1
IntFunction<Character> toHex = hexDigits::charAt;  2
// al posto di i -> hexDigits.charAt(i)           3
                                                    4
System.out.println(toHex.apply(15)); // stampa 'F' 5
```

Metodo statico

```
Supplier<Double> randomNum = Math::random;         1
// al posto di () -> Math.random()                 2
                                                    3
System.out.println(randomNum.get()); // stampa un numero decimale casuale 4
```

METHOD REFERENCE (CONT.)

Riferimento a metodo d'istanza

```
Function<String, Integer> strLen = String::length;           1
// al posto di str -> str.length()                          2
                                                                3
String[] arrStr = {"two", "four"}; // stampa '3' e '4'       4
for (String str : arrStr) System.out.println(strLen.apply(str)); 5
```

Costruttore

```
Function<String, Person> personBuilder = Person::new;        1
// al posto di str -> new Person(str)                        2
                                                                3
Person sam = personBuilder.apply("Sam")                       4
System.out.println(sam); // stampa 'Sam'                     5
```

STREAM



STREAM API

Il package `java.util.stream` (Java 8) contiene classi per elaborare stream

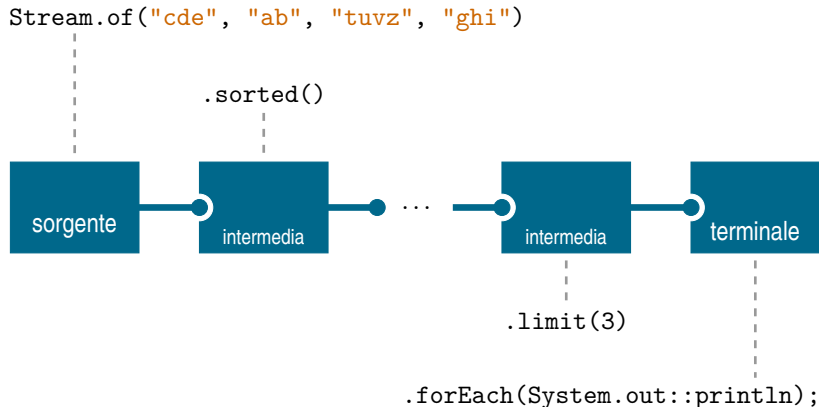
Uno `Stream<T>` è una sequenza di elementi `T` provenienti da una sorgente

- ▶ Tale sequenza supporta operazioni di elaborazione dati
- ▶ Le quali sono definite mediante interfacce funzionali

Gli stream forniscono nativamente

- ▶ Pipelining
- ▶ Iterazione interna
- ▶ Parallelizzazione

STREAM API (CONT.)



SORGENTI

Stream da una lista variabile di argomenti, con `Stream.of(T... elems)`

```
Stream<String> stream = Stream.of("a", "b", "c");
```

1

Stream da un array, con `Arrays.stream(T[] arr)`

```
String[] arr = new String[]{"a", "b", "c"};  
Stream<String> stream = Arrays.stream(arr);
```

1

2

Stream da una collezione, con `Collection.stream()`

```
Set<String> set = Set.of("a", "b", "c");  
Stream<String> stream = set.stream();
```

1

2

UNWRAP DEGLI STREAM

List da uno stream, con `Stream.toList()`

```
List<Integer> list = Stream.of(1, 2, 2, 3).toList();    1
// list contiene [1, 2, 2, 3]                          2
```

Set da uno stream, con `Stream.collect(Collector<T, A, R> collector)`

```
                collectors.toSet());    1
// set contiene [1, 2, 3]              2
```

Array da uno stream, con `Stream.toArray(IntFunction<T[]> generator)`

```
                r[]::new);    1
// arr contiene [1, 2, 2, 3]  2
```

GENERAZIONE DI STREAM

Operazione che applica ripetutamente un operatore unario su un seed per creare uno stream

```
Stream<T> iterate(T seed, UnaryOperator<T> generator)
```

Ricorda che un `UnaryOperator<T>` accetta un valore `T` e produce un valore `T`

- ▶ Genera un numero potenzialmente infinito di valori
- ▶ Viene restituito un nuovo stream con gli elementi generati

GENERAZIONE DI STREAM (CONT.)

Generazione infinita

// valutazione lazy

```
Stream<Double> stream = Stream.iterate(Math.random(), d -> Math.random());  
// genera una sequenza illimitata di numeri decimali casuali  
  
// equivalente, si aspetta un Supplier<T>  
Stream<Double> stream = Stream.generate(Math::random); // il calcolo si arresta (valutazione lazy)  
List<Double> list = stream.toList(); // il calcolo non terminerà mai
```

Generazione controllata

```
Stream<Integer> stream = Stream.iterate(0, i -> i + 1).limit(10);  
List<Integer> list = stream.toList();  
// genera i primi dieci numeri naturali [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

OPERAZIONI INTERMEDIE E TERMINALI

Le operazioni intermedie restituiscono un altro oggetto `Stream<T>`, come la sorgente

- ▶ Non modificano la sorgente dello stream
- ▶ Possono essere concatenate, usando la dot-notation

```
String[] arr = {"a", "c", "b", "c"};           1
Stream<String> distinct = Arrays.stream(arr).distinct().sorted(); 2
long count = Arrays.stream(arr).distinct().count(); 3
```

Le operazioni terminali restituiscono un risultato, di un tipo specifico

- ▶ È consentita solo una singola operazione terminale, se presente

OPERAZIONI INTERMEDIE E TERMINALI (CONT.)

Le operazioni intermedie restituiscono un altro oggetto `Stream<T>`, come la sorgente

- ▶ Non modificano la sorgente dello stream
- ▶ Tuttavia gli stream possono essere consumati una sola volta

```
String[] arr = {"a", "c", "b", "c"};           1
Stream<String> stream = Arrays.stream(arr);    2
Stream<String> distinct = stream.distinct().sorted(); 3
long count = stream.distinct().count();       4
```

Questo produrrà un'eccezione a runtime!

FILTER

Operazione intermedia che applica un predicato su ciascun elemento dello stream

```
Stream<T> filter(Predicate<T> predicate)
```

Ricorda che un `Predicate<T>` rappresenta un controllo booleano su valori `T`

- ▶ Il predicato viene verificato su tutti gli elementi dello stream
- ▶ Viene restituito un nuovo stream con gli elementi che soddisfano il predicato

FILTER (CONT.)

Con una lambda

```
String[] arr = {"abc", "cde", "ecd", "cba"};           1
Stream<String> stream = Arrays.stream(arr).filter(s -> s.contains("d")); 2
// stream contiene "cde" ed "ecd"                        3
```

Con un method reference

```
Character[] arr = {'a', '3', '7', 'd'};                1
Stream<Character> stream = Arrays.stream(arr).filter(Character::isDigit); 2
// stream contiene '3' e '7'                            3
```

FILTRAGGIO INTERMEDIO

Scarta gli elementi duplicati: `Stream<T> distinct()`

Restituisce i primi n elementi: `Stream<T> limit(int n)`

Scarta i primi n elementi: `Stream<T> skip(int n)`

Ordina gli elementi secondo l'ordinamento naturale: `Stream<T> sorted()`

- ▶ Qui, `T` deve implementare `Comparable`
- ▶ Oppure secondo un comparatore: `Stream<T> sorted(Comparator<T> comp)`

MAP

Operazione intermedia che applica una trasformazione sugli elementi dello stream

```
Stream<R> map(Function<T,R> mapper)
```

Ricorda che una `Function<T,R>` trasforma valori `T` in valori `R`

- ▶ La trasformazione viene applicata su tutti gli elementi dello stream
- ▶ Viene restituito un nuovo stream con gli elementi trasformati

MAP (CONT.)

Con una lambda

```
String[] arr = {"abc", "cdef", "ec", "cba"};           1
Stream<Integer> stream = Arrays.stream(arr).map(s -> s.length()); 2
// stream contiene 3, 4, 2 e 3                             3
```

Con un method reference

```
String[] arr = {"aBc", "cd7f", "eC", "2Ba"};           1
Stream<String> stream = Arrays.stream(arr).map(String::toUpperCase); 2
// stream contiene "ABC", "CD7F", "EC" e "2BA"           3
```

VERIFICA DI PREDICATI

Operazioni terminali che applicano un predicato all'intero stream

Almeno un elemento dello stream soddisfa il predicato

```
boolean anyMatch(Predicate<T> predicate)
```

Tutti gli elementi dello stream soddisfano il predicato

```
boolean allMatch(Predicate<T> predicate)
```

Nessun elemento dello stream soddisfa il predicato

```
boolean noneMatch(Predicate<T> predicate)
```

VERIFICA DI PREDICATI (CONT.)

Trova il primo elemento dello stream (eventualmente soddisfacente un predicato)

```
Optional<T> findFirst()
```

Trova un elemento dello stream (eventualmente soddisfacente il predicato)

```
Optional<T> findAny()
```

Viene restituito un optional vuoto se lo stream è vuoto

- Per esempio, quando un filtraggio precedente non produce elementi

OPTIONAL

Un optional rappresenta un valore potenziale

```
Integer[] arr = {-1, 4, -5, 0, 3}; 1
Optional<Integer> opt = Arrays.stream(arr).filter(e -> e <= 0).findAny(); 2
3
// controlla se un elemento è stato trovato 4
if (opt.isPresent()) System.out.println(opt.get()); 5
else System.out.println(0); 6
7
// versione equivalente, più compatta 8
System.out.println(opt.orElse(0)); 9
```


ITERAZIONE SU STREAM

Operazione terminale che itera sugli elementi dello stream

- ▶ L'azione viene eseguita su tutti gli elementi dello stream
- ▶ Lo stream viene consumato

// non può essere riutilizzato

```
void forEach(Consumer<T> action)
```

```
// stampa uno stream a schermo 1
Integer[] arr = {-1, 4, -5, 0, 3}; 2
Arrays.stream(arr).filter(e -> e <= 0).forEach(System.out::println); 3
4
// conta gli elementi di uno stream 5
System.out.println(Arrays.stream(arr).filter(e -> e <= 0).count()); 6
```

FLAT MAP

Operazione intermedia che estrae uno stream da ciascun elemento dello stream

```
Stream<R> flatMap(Function<T,Stream<R>> mapper)
```

Qui `Function<T,Stream<R>>` trasforma valori `T` in stream di `R`

- ▶ La trasformazione viene applicata su tutti gli elementi dello stream
- ▶ Viene restituito un nuovo stream che concatena tutti gli stream risultanti
- ▶ Tipicamente `T` rappresenta valori multipli (ad es. una collezione)

FLAT MAP (CONT.)

Per gli array

```
Byte[][] arr = { { 1, 2, 3}, {4, 5, 6}, {7, 8, 9} };           1
Stream<Byte> stream = Arrays.stream(arr).flatMap(r -> Arrays.stream(r)); 2
// stream contiene 1, 2, 3, 4, 5, 6, 7, 8 e 9                      3
```

Per le collezioni

```
List<List<Integer>> list = new LinkedList<>();                  1
list.add(new LinkedList<>()); list.add(new LinkedList<>());    2
list.get(0).add(1); list.get(0).add(3);                        3
list.get(1).add(7); list.get(1).add(9);                        4
Stream<Integer> stream = list.stream().flatMap(Collection::stream); 5
// stream contiene 1, 3, 7 e 9                                   6
```

FLAT MAP (CONT.)

Di nuovo concatenazione

```
Integer[][] arr = { { 1, -2, 3}, {4, -5, -6}, {-7, -8, 9} };      1
Arrays.stream(arr)                                              2
    .flatMap(Arrays::stream)                                    3
    .filter(i -> i > 0)                                         4
    .map(i -> String.format("%d è positivo", i))               5
    .forEach(System.out::println);                             6

//                                                                7
// output:                                                       8
// '1 è positivo'                                                9
// '3 è positivo'                                              10
// '4 è positivo'                                              11
// '9 è positivo'                                              12
```

DOMANDE E RISPOSTE

